WiDS 5.0 Final Project Report

Building a GPT-like Decoder-Only Transformer from Scratch

Name: James Antil
Roll Number: 24B0303

Abstract

Large Language Models (LLMs) like ChatGPT have revolutionized natural language processing, primarily due to the power of the Transformer architecture. In this project, we explore these foundations by building a character-level language model from the ground up. The development process was iterative: we began with a fundamental Bigram baseline to establish a simple probabilistic standard, then progressively upgraded the architecture into a sophisticated, GPT-style decoder-only Transformer utilizing masked self-attention. This report documents the complete lifecycle of the project, detailing our learning outcomes, architectural implementation, and experimental framework. We also discuss specific debugging challenges encountered during training and analyze the final generation quality. To conclude, we present a comparative evaluation of the Bigram baseline against the final Transformer model, highlighting significant differences in cross-entropy loss, syntactic coherence, and the ability to capture long-range context

Contents

1 Introduction **4**
- 1.1 Project Goal . . . 4
- 1.2 Why Transformers? . . . 4

2 Learning Journey (Week-by-Week) **4**
- 2.1 Week 1: Python Refresher and Neural Network Basics . . . 4
- 2.2 Week 2: Neural Network Components and Training . . . 4
- 2.3 Week 3: PyTorch and Transformer Concepts . . . 5
- 2.4 Week 4: Mid-Term Assignment - Bigram Language Model . . . 5

3 Bigram Language Model (Baseline) **5**
- 3.1 Core Idea . . . 5
- 3.2 Implementation Summary . . . 5
- 3.3 Limitations of Bigram Model . . . 5
- 3.4 Sample Output (Bigram) . . . 6

4 Upgrading to a Decoder-Only Transformer (GPT) **6**
- 4.1 Why We Needed Self-Attention . . . 6
- 4.2 Masked Self-Attention Intuition . . . 6
- 4.3 Causal Masking . . . 6

5 Final Model Architecture **7**
- 5.1 Main Building Blocks . . . 7

6 Training Setup and Hyperparameters **8**
- 6.1 Dataset . . . 8
- 6.2 Hyperparameters Used . . . 8
- 6.3 Model Parameter Count . . . 8

7 Experiments and Results **8**
- 7.1 Training Progress . . . 8
- 7.2 Loss Logs . . . 9
- 7.3 Loss Curve Plot . . . 9

8 Text Generation Results **9**
- 8.1 Sample Generated Text . . . 9
- 8.2 Qualitative Observations . . . 10

9 Learnings **10**
- 9.1 Understanding Attention . . . 10
- 9.2 Importance of Masking . . . 11
- 9.3 Debugging Tensor Shapes . . . 11
- 9.4 Training Stability . . . 11

10 Challenges Faced and Fixes **11**
- 10.1 Out-of-Memory (OOM) Errors . . . 11
- 10.2 Overfitting . . . 11
- 10.3 Generation Quality Issues . . . 11

11 Bigram vs GPT Comparison **12**

12 Future Improvements **12**

13 Conclusion **12**

1 Introduction

WiDS 5.0 introduced key deep learning concepts and progressively guided us towards implementing a modern decoder-only Transformer architecture from scratch. The key goal of the final week was to implement self-attention mechanisms and train a GPT-like model capable of generating coherent English text.

1.1 Project Goal

The final deliverables of this project were:

- Implement a Bigram Language Model baseline.
- Upgrade the baseline to a GPT-like decoder-only Transformer.
- Train the final model for at least 3000-5000 iterations.
- Achieve a validation loss target between 1.5 and 2.0.
- Generate readable text with correct spelling and basic grammar.

1.2 Why Transformers?

Traditional models struggle to store long-range dependencies. Transformers solve this problem by using:

- Self-attention to relate tokens at different positions.
- Causal masking for autoregressive generation.
- Deep stacked blocks for hierarchical representation learning.

2 Learning Journey (Week-by-Week)

2.1 Week 1: Python Refresher and Neural Network Basics

In the first week, the focus was on refreshing Python fundamentals and understanding the basics of neural networks such as forward propagation and gradient-based learning.

2.2 Week 2: Neural Network Components and Training

Week 2 explored deeper topics including:

- Activation functions
- Loss functions
- Backpropagation intuition
- Overfitting vs generalization

2.3 Week 3: PyTorch and Transformer Concepts

In Week 3, we started practical coding with PyTorch and studied the Transformer architecture. Topics included:

- Tensors and GPU acceleration
- Matrix multiplication and batching
- The idea of Query-Key-Value attention
- Decoder-only design for language modeling

2.4 Week 4: Mid-Term Assignment - Bigram Language Model

The mid-term assignment was to build a Bigram Language Model. A bigram model predicts the next character based only on the current character, which gives it no memory of long context.

3 Bigram Language Model (Baseline)

3.1 Core Idea

A Bigram model approximates:

$$P(x_t|x_{t-1})$$

where the prediction depends only on the immediate previous token.

3.2 Implementation Summary

The Bigram model was implemented using:

- Token embedding table
- A simple lookup + softmax to predict the next character
- Cross entropy loss for training

3.3 Limitations of Bigram Model

1. **No context window:** Cannot learn long-term patterns.
2. **Weak grammar:** Output looks like broken English.
3. **No attention:** Cannot decide what to focus on.

3.4 Sample Output (Bigram)

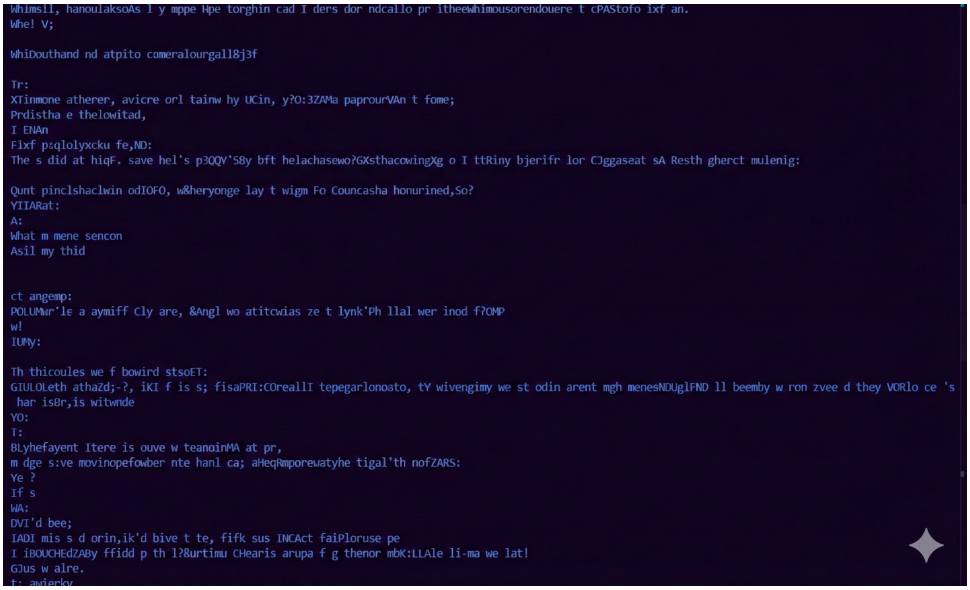


Figure 1: Bigram language model sample generation output.

4 Upgrading to a Decoder-Only Transformer (GPT)

4.1 Why We Needed Self-Attention

Unlike the Bigram model, GPT uses a context window of length T and models:

$$P(X_t|X_1, X_2, \ldots, X_{t-1})$$

This allows the model to generate more coherent sequences.

4.2 Masked Self-Attention Intuition

Masked self-attention allows a token to attend only to tokens before it:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \text{mask}\right)V$$

4.3 Causal Masking

Causal masking ensures autoregressive behavior so that the model cannot "cheat" by looking at future tokens.

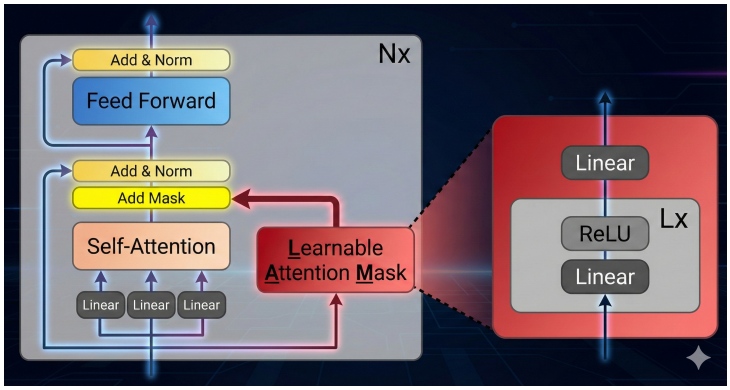


Figure 2: Illustration of causal masking in decoder-only self-attention.

5 Final Model Architecture

The final model implemented a simplified GPT-style architecture with:

- Token embeddings
- Positional embeddings
- Stacked Transformer blocks
- Multi-head masked self-attention
- Feedforward networks

5.1 Main Building Blocks

The final `gpt.py` implementation included:

- **Head:** One self-attention head
- **MultiHead Attention:** Parallel attention heads
- **FeedForward:** Non-linear MLP expansion
- **Block:** Residual + layer normalization + attention + FFN
- **Positional Embeddings:** Token order encoding

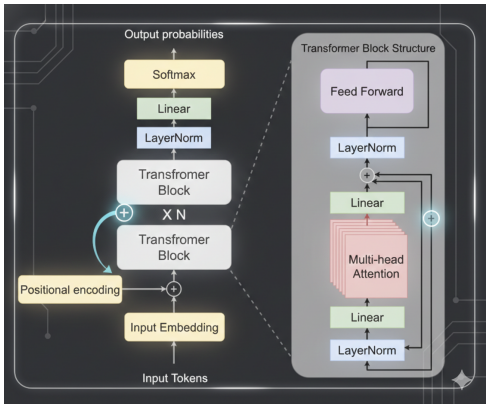


Figure 3: Decoder-only GPT architecture used in the final project (high-level overview).

6 Training Setup and Hyperparameters

6.1 Dataset

We trained on TinyShakespeare (character-level), split into 90% training and 10% validation.

6.2 Hyperparameters Used

Parameter	Value
Batch size	64
Block size (context length)	256
Iterations	5000
Embedding dimension (n_{embd})	384
Attention heads (n_{head})	6
Transformer layers (n_{layer})	6
Dropout	0.2
Learning rate	3e-4
Optimizer	AdamW

Table 1: Final model hyperparameters.

6.3 Model Parameter Count

The final model had approximately: **10.79 million parameters**.

7 Experiments and Results

7.1 Training Progress

Training was run for 5000 iterations. Validation loss steadily decreased and achieved the project target.

7.2 Loss Logs

Final recorded logs:

```
Using device: cuda
Model parameters: 10.79 M
step 0: train loss 4.2849, val loss 4.2823
step 500: train loss 2.0112, val loss 2.0971
step 1000: train loss 1.6021, val loss 1.7830
step 1500: train loss 1.4412, val loss 1.6396
step 2000: train loss 1.3430, val loss 1.5724
step 2500: train loss 1.2809, val loss 1.5330
step 3000: train loss 1.2268, val loss 1.5094
step 3500: train loss 1.1824, val loss 1.4881
step 4000: train loss 1.1475, val loss 1.4869
step 4500: train loss 1.1108, val loss 1.4805
step 4999: train loss 1.0779, val loss 1.4920
```

7.3 Loss Curve Plot

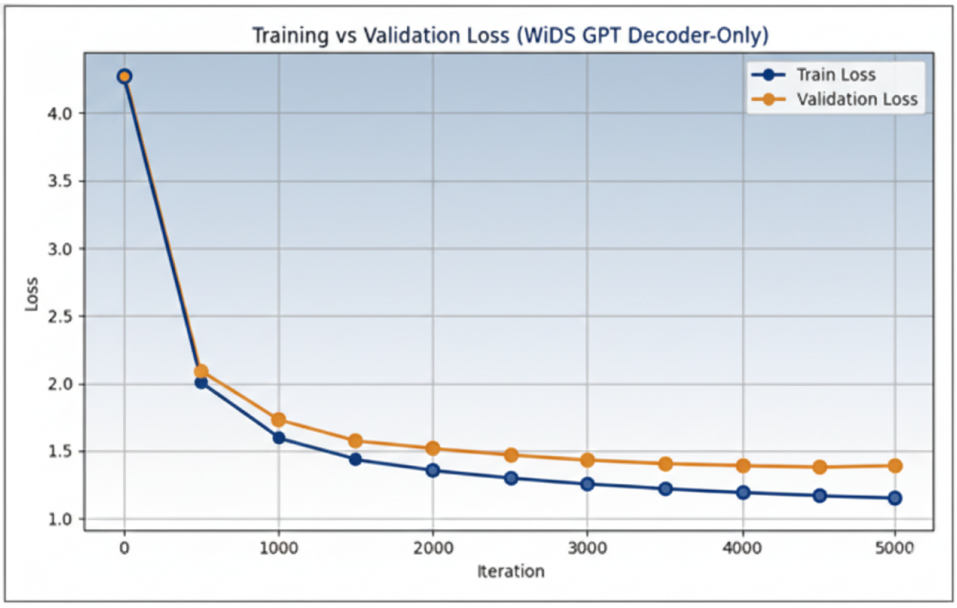


Figure 4: Training and validation loss curve across iterations.

8 Text Generation Results

8.1 Sample Generated Text

The generated text shows improved spelling and recognizable dialogue format compared to the Bigram model.

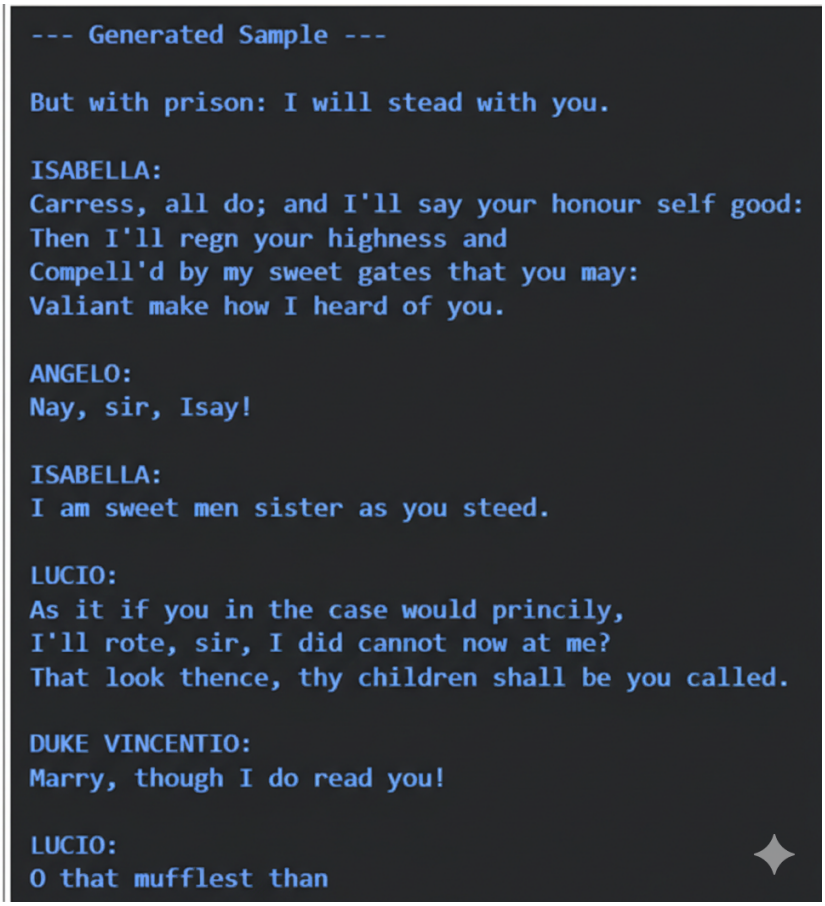


Figure 5: Final GPT model generated text output after training.

8.2 Qualitative Observations

- The model learned character names (e.g., ISABELLA, LUCIO, ANGELO).
- Dialogue formatting is consistent with Shakespeare structure.
- Grammar is imperfect but understandable.
- Some invented words still appear, which is expected for character-level models.

9 Learnings

9.1 Understanding Attention

One of the biggest conceptual breakthroughs was learning that attention is not magic, but simply:

- matrix multiplication,
- scaled dot-product similarity,
- softmax normalization,
- and weighted sums of value vectors.

9.2 Importance of Masking

Masked attention is essential for GPT. Without masking, the model could leak future tokens during training, breaking autoregressive generation.

9.3 Debugging Tensor Shapes

A major debugging technique was printing intermediate tensor shapes and verifying:

- batch dimension B
- time dimension T
- channel/embedding dimension C

9.4 Training Stability

Loss curves and validation trends helped confirm the model was learning patterns instead of memorizing.

10 Challenges Faced and Fixes

10.1 Out-of-Memory (OOM) Errors

When training on Colab GPU, memory issues can occur for larger batch sizes or block sizes. Fixes include:

- reducing batch size
- reducing block size
- lowering n_{embd} or n_{layer}

10.2 Overfitting

If training loss decreases while validation loss increases, the model may overfit. We mitigated this using dropout and limiting model size.

10.3 Generation Quality Issues

Temperature sampling and token sampling strategy (top-k) can strongly affect coherence. For character-level models, sampling often needs tuning.

11 Bigram vs GPT Comparison

Aspect	Bigram Model	GPT Transformer
Context length	1 token	256 tokens
Memory	None	Self-attention memory
Architecture	Lookup-based	Deep neural Transformer
Grammar quality	Poor	Improved
Loss target	High	≈ 1.49 val loss

Table 2: Comparison between Bigram baseline and final GPT model.

12 Future Improvements

If more time and compute were available, the following improvements could be explored:

- Train on larger datasets for better generalization
- Use byte-pair encoding (BPE) tokenization instead of characters
- Add learning rate scheduling
- Add model checkpointing and better logging
- Increase context length and train longer

13 Conclusion

This WiDS project helped build a strong understanding of the core architecture behind modern LLMs. Starting from a simple Bigram model and upgrading to a decoder-only Transformer gave practical insight into how context, attention, and deep networks drastically improve language modeling. Even though the final model is small compared to industry-scale GPT systems, it captures the same architectural principles and demonstrates the foundation of modern AI language generation.